

Task 1 — Find Maximum Value

```
ALGORITHM FindMax(A[a1..an], k)
// Finds the maximum value in an array
// Input: Array A[a1..an], num k
// Output: Maximum value in A
```

```
max ← A[0]

for i ← 1 to n do
    if A[i] > max
        max ← A[i]

return max
```

Explanation

The algorithm finds the largest element in the array. It starts by assuming that the first element is the maximum, then compares every other element with **max**. If a larger element is found, **max** is updated. Finally, the maximum value is returned.

Task 2 — Calculate Average

```
ALGORITHM CalculateAverage(A[a1..an], k)
// Calculates the average of an array's numbers
// Input: Array A[a1..an], num k
// Output: Average value of A
```

```
sum ← 0

for i ← 1 to k do
    sum ← sum + A[i]

average ← sum / k

return average
```

Explanation

The algorithm calculates the average of the first k elements of the array. It adds all selected elements to sum , divides the total by k , and returns the resulting average.

1.1.4 — Square Root

```
ALGORITHM SquareRoot(n)
// Computes the square root of a positive integer n
// Input: A positive integer n
// Output: An approximation of  $\sqrt{n}$ 
```

```
x ← n

y ← (x + n / x) / 2

while y < x do
    x ← y
    y ← (x + n / x) / 2

return x
```

Explanation

The algorithm approximates the square root of n using repeated averaging. It starts with $x = n$ and repeatedly calculates a better approximation using $(x + n/x) / 2$. The process stops when the new approximation is no longer smaller than the current value.

1.1.6 a — Euclid's GCD Algorithm

```
ALGORITHM EuclidGCD(m, n)
// Finds the greatest common divisor of two integers
// Input: Two positive integers m and n
// Output: Greatest common divisor of m and n
```

```
while n ≠ 0 do
    r ← m mod n
    m ← n
    n ← r
```

```
return m
```

Explanation

The algorithm finds the greatest common divisor (GCD) of m and n . It repeatedly replaces m and n with n and the remainder of $m \div n$. When n becomes 0, m is the GCD.

1.1.6 b — Compare GCD Algorithms

```
ALGORITHM CompareGCDAlgorithms(m, n)
// Estimates how many times faster Euclid's algorithm is
// Input: Two positive integers m and n
// Output: Estimated speedup

t ← min(m, n)
count ← 0

while t ≥ gcd(m, n) do
    count ← count + 1
    t ← t - 1

euclidSteps ← 0

while n ≠ 0 do
    r ← m mod n
    m ← n
    n ← r
    euclidSteps ← euclidSteps + 1

speedup ← count / euclidSteps

return speedup
```

Explanation

This algorithm compares two approaches for finding the GCD. The first approach checks consecutive integers and counts how many checks are needed. The second approach uses Euclid's algorithm and counts its iterations. The ratio $\text{count} / \text{euclidSteps}$ estimates how many times faster Euclid's algorithm is.

```
min(31415, 14142) = 14142
gcd(31415, 14142) = 1
```

```
Consecutive-integers checks = 14142
Euclid's algorithm steps = 9
```

```
speedup  $\leftarrow$  14142 / 9
```

```
return 1571
```

Explanation

For these values, the consecutive-integers method requires 14,142 checks, while Euclid's algorithm requires only 9 steps. Their ratio is approximately 1571, showing a large difference in the number of operations.

1.1.7 — Euclid's GCD Algorithm

```
ALGORITHM EuclidGCD(m, n)
// Finds the GCD using Euclid's algorithm
// Input: Two positive integers m and n
// Output: gcd(m, n)

while n  $\neq$  0 do
    r  $\leftarrow$  m mod n
    m  $\leftarrow$  n
    n  $\leftarrow$  r

return m
```

Explanation

This is another presentation of Euclid's algorithm. It repeatedly calculates the remainder of m divided by n . The values are shifted until the remainder becomes zero. The remaining value of m is the GCD.

1.1.8 — Euclid's GCD with Swapping

ALGORITHM EuclidGCD(m, n)

// Finds the greatest common divisor of two positive integers

// Input: Two positive integers m and n, where $m < n$

// Output: Greatest common divisor of m and n

if $m < n$

 swap(m, n)

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Explanation

This version first checks whether m is smaller than n . If so, it swaps them so that m is the larger value. Then it applies the standard Euclidean algorithm until n becomes zero.

1.1.9 a — Euclid's GCD for Restricted Integers

ALGORITHM EuclidGCD(m, n)

// Finds the GCD using Euclid's algorithm

// Input: Two integers $1 \leq m, n \leq 10$

// Output: Greatest common divisor of m and n

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Explanation

The algorithm calculates the GCD using Euclid's method. The input values are restricted to the range $1 \leq m, n \leq 10$. The algorithm repeatedly replaces the pair with the divisor and remainder until the remainder becomes zero.

1.1.9 b — Euclid's GCD

```
ALGORITHM EuclidGCD(m, n)
// Finds the GCD using Euclid's algorithm
// Input: Two integers  $1 \leq m, n \leq 10$ 
// Output: Greatest common divisor of m and n

while  $n \neq 0$  do
     $r \leftarrow m \bmod n$ 
     $m \leftarrow n$ 
     $n \leftarrow r$ 

return m
```

Explanation

This task uses the standard Euclidean algorithm to calculate the GCD. The remainder is calculated repeatedly, and the process ends when n becomes zero. The final value of m is the GCD.

1.2.4 — Quadratic Equation Roots

```
ALGORITHM QuadraticRoots(a, b, c)
// Finds the real roots of a quadratic equation  $ax^2 + bx + c = 0$ 
// Input: Real coefficients a, b, and c
// Output: The real roots of the equation (if any)

if  $a = 0$  then
    if  $b = 0$  then
        if  $c = 0$  then
            return "Infinitely many roots"
        else
            return "No roots"
    else
        return  $-c / b$ 
else
     $D \leftarrow b^2 - 4ac$ 
```

```

if D < 0 then
    return "No real roots"
else if D = 0 then
    return -b / (2a)
else
    x1 ← (-b + sqrt(D)) / (2a)
    x2 ← (-b - sqrt(D)) / (2a)
    return x1, x2

```

Explanation

The algorithm finds the roots of $ax^2 + bx + c = 0$. First, it checks whether $a = 0$, because in that case the equation is linear or has no/infininitely many solutions. Otherwise, it calculates the discriminant $D = b^2 - 4ac$.

- If $D < 0$, there are no real roots.
 - If $D = 0$, there is one real root.
 - If $D > 0$, there are two real roots.
-

1.2.5 — Decimal to Binary Conversion

1. Divide the number n by 2.
2. Record the remainder (which is either 0 or 1).
This remainder is the next binary digit (bit), starting from the least significant bit (rightmost).
3. Replace n with the quotient of the division (integer division).
4. Repeat steps 1–3 until the quotient becomes 0.
5. Read the remainders in reverse order
(from last obtained to first) to get the binary representation.

```

// Converts a positive decimal integer n into its binary representation
// Input: A positive integer n
// Output: The binary representation of n as a string of bits

```

ALGORITHM BinaryRepresentation(n)

```
bits ← empty string
```

```

while n > 0 do
    r ← n mod 2

```

```
bits ← r + bits
n ← n / 2    // integer division

return bits
```

Explanation

The algorithm converts a decimal integer into binary. It repeatedly divides the number by 2 and records the remainder, which is always 0 or 1. The remainders are added to the beginning of the string so that the final result is in the correct binary order.

1.2.6 — ATM Cash Dispensing

```
ALGORITHM ATMDispenseCash(card, pin, amount)
// Dispenses cash from an ATM
// Input: Card, PIN, and requested amount
// Output: Cash or an error message
```

```
Insert card
```

```
if card is valid then
    Enter PIN
```

```
    if PIN is correct then
        Enter amount
```

```
        if amount ≤ account balance then
```

```
            if amount ≤ ATM cash available then
                Dispense cash
                Update account balance
                return "Transaction successful"
            else
                return "Insufficient cash in ATM"
```

```
        else
            return "Insufficient account balance"
```

```
    else
        return "Incorrect PIN"
```

```
else
```

```
return "Invalid card"
```

Explanation

This algorithm describes the logical steps of an ATM withdrawal. First, the card is checked. Then the PIN is verified. If the PIN is correct, the requested amount is compared with both the account balance and the cash available in the ATM. If both conditions are satisfied, cash is dispensed and the account balance is updated.

1.2.9 — Minimum Distance Between Array Elements

```
ALGORITHM MinDistance(A[0..n - 1])  
// Finds the minimum distance between two elements in an array  
// Input: Array A[0..n - 1] of numbers  
// Output: Minimum distance between two of its elements
```

```
dmin ← ∞  
  
for i ← 0 to n - 1 do  
    for j ← 0 to n - 1 do  
        if i ≠ j and |A[i] - A[j]| < dmin  
            dmin ← |A[i] - A[j]|  
  
return dmin
```

Explanation

The algorithm finds the smallest absolute difference between any two different elements of the array. It compares every pair of elements, calculates $|A[i] - A[j]|$, and updates **dmin** whenever a smaller distance is found. Finally, it returns the minimum distance.